

8/7/26

H.W

Define a structure in C & CPP. This structure should hold the variables for student type.

- 1) Compute the size of the structure manually.
- 2) Use size of function and display the size cross check if both are same & not justify the answer.

A structure in C & C++ is a user defined data type used to store related data items of different types under one name.

Structure in C:

```
#include <stdio.h>
```

```
struct student {
```

```
    int rollno;
```

```
    char name[10];
```

```
    float marks;
```

```
    char grade;
```

```
};
```

```
int main() {
```

```
    struct student
```

```
    S1 = {101, "Triveni", 92.5}
```

```
    printf("Rollno: %d\n", S1.rollno);
```

```
    printf("Name: %s\n", S1.name);
```

```
    printf("Marks: %.2f\n", S1.marks);
```

```
    printf("Grade: %c\n", S1.grade);
```

```
    return 0;
```

```
}
```

Structure in C++

```
#include <iostream>
```

```
using namespace std;
```

```
struct student {
```

```
    int rollno;
```

```
    char name[10];
```

```
    float marks;
```

```
    char grade;
```

```
};
```

```
int main() {
```

```
    student S1 = {101, "Triveni",
```

```
    92.5, 'A'};
```

```
    cout << "Rollno: " << S1.rollno << endl;
```

```
    cout << "Name: " << S1.name << endl;
```

```
    cout << "Marks: " << S1.marks << endl;
```

```
    cout << "Grade: " << S1.grade << endl;
```

```
    return 0;
```

```
}
```


Manually we can take:

int - 4 bytes

char[10] - 10 bytes

float - 4 bytes

char - 1 byte

19 bytes

Size of (student) will give 32 bytes commonly.

Justification:

The sizes are usually not the same because of structure padding.

Structure padding means extra unused bytes added by the compiler inside & at the end of a structure

```
#include <stdio.h>
```

```
struct Student {
```

```
    int rollno;
```

```
    char name[20];
```

```
    float marks;
```

```
    char grade;
```

```
};
```

```
int main ( ) {
```

```
    struct Student S;
```

```
    int manual Size;
```

```
    int actual Size;
```

manual Size = Size of (int) + (20 * size of (char)) +
Size of (float) + size of (char);


```
actualSize = Size of (s);
```

```
printf ("Manual Size = %d bytes\n", manualSize);
```

```
printf ("Size using Size of - %d bytes\n", actualSize);
```

```
if manualSize == actualSize
```

```
printf ("Both sizes are same\n");
```

```
else {
```

```
printf ("Both sizes are not same\n");
```

```
return 0;
```

```
}
```

Sum of n numbers using recursion:

```
if (n == 0)
```

```
return 0;
```

```
else
```

```
return n + sum(n-1);
```

```
}
```

```
int main()
```

```
{
```

```
int n, result;
```

```
printf ("Enter a number:");
```

```
scanf ("%d", &n);
```

```
result = sum(n);
```

```
printf ("Sum of first %d natural numbers
```

```
- %d", n, result);
```

```
return 0;
```